

PostgreSQL 18 Logical Replication — Hands-on Lab

Publisher: live Patroni primary on acid-postgres-ha. Subscriber: isolated acid-recovery-test-0 (recoverydb). Goal: copy one table with publication/subscription, prove live INSERT, then clean up.

Do not create a subscription on acid-postgres-ha-1 or -2 pointing at the same cluster. Those pods already use physical streaming. Logical replication is an outbound copy to another database.

Architecture (what you are building)

App writes primary → WAL (wal_level=logical) → PUBLICATION app_pub filters tables → logical slot + pgoutput → Service acid-postgres-ha → SUBSCRIPTION app_sub on recovery-test → apply rows.

Step 0 — Variables and recovery pod

```
NS=postgres-ha
PRIMARY=$(for p in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  v=$(kubectl -n "$NS" exec "$p" -- psql -U postgres -Atqc "select pg_is_in_recovery();")
  if [ "$v" = "f" ]; then echo "$p"; break; fi
done)
echo "PRIMARY=$PRIMARY"
kubectl -n "$NS" get pod acid-recovery-test-0
SUB=acid-recovery-test-0
```

If acid-recovery-test-0 is Missing, apply Lab 8 acid-recovery-test.yaml and wait until the pod is Ready.

Step 1 — Confirm wal_level=logical

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "SHOW wal_level;"
```

If the result is replica, logical decoding is off. Set wal_level: logical on the Zalando/Patroni cluster and roll pods, then restart this lab. Physical HA still works at replica; publications do not.

Step 2 — Demo table on the publisher (banking)

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c "
CREATE TABLE IF NOT EXISTS public.logical_demo (
  id int PRIMARY KEY,
  note text,
  updated_at timestamptz DEFAULT now()
);
INSERT INTO public.logical_demo (id, note) VALUES (1, 'seed-on-primary')
ON CONFLICT (id) DO NOTHING;
SELECT * FROM public.logical_demo;
"
```

A primary key is required so UPDATE/DELETE can find the row on the subscriber.

Step 3 — Replication role and publication

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "
DO \$\$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'repl_user') THEN
    CREATE ROLE repl_user WITH REPLICATION LOGIN PASSWORD 'ChangeMe-Lab-Only';
  END IF;
END\$\$;
GRANT CONNECT ON DATABASE banking TO repl_user;
GRANT USAGE ON SCHEMA public TO repl_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO repl_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO repl_user;
DROP PUBLICATION IF EXISTS app_pub;
CREATE PUBLICATION app_pub FOR TABLE public.logical_demo;
SELECT * FROM pg_publication_tables;
"
```

Step 4 — Same table on the subscriber (empty)

Logical replication does not ship CREATE TABLE. Create the table first.

```
kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c "
CREATE TABLE IF NOT EXISTS public.logical_demo (
  id int PRIMARY KEY,
  note text,
  updated_at timestamptz DEFAULT now()
);
SELECT count(*) AS subscriber_rows FROM public.logical_demo;
"
```

Step 5 — Create the subscription

Connect to the primary Service so failover still works. This example uses user postgres if the Service already allows it from in-cluster pods. Prefer repl_user once pg_hba/operator users allow that role.

```
kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c "
DROP SUBSCRIPTION IF EXISTS app_sub;
CREATE SUBSCRIPTION app_sub
  CONNECTION 'host=acid-postgres-ha.postgres-ha.svc.cluster.local port=5432 user=postgres dbname=banking'
  PUBLICATION app_pub
  WITH (copy_data = true, create_slot = true, enabled = true);
"
```

If host DNS fails, try host=acid-postgres-ha. If authentication fails, switch the CONNECTION user to repl_user and fix pg_hba / operator users.

Step 6 — Wait for initial copy (state r)

```
kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c \
"SELECT srrelid::regclass AS table, srsubstate FROM pg_subscription_rel;"

kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c \
"SELECT subname, pid, received_lsn FROM pg_stat_subscription;"

kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c \
"SELECT slot_name, plugin, active, confirmed_flush_lsn
FROM pg_replication_slots WHERE slot_type='logical';"

kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c \
"SELECT * FROM public.logical_demo;"
```

srsubstate: i = initializing, d/s = copy done, r = ready (streaming). Expect row id=1 seed-on-primary on the subscriber after r.

Step 7 — Prove live DML

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c \
"INSERT INTO public.logical_demo(id, note) VALUES (2, 'after-subscribe');"

sleep 3

kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c \
"SELECT * FROM public.logical_demo ORDER BY id;"
```

Pass: subscriber shows ids 1 and 2. That row did not arrive via Patroni streaming; recovery-test is a separate cluster.

Step 8 — Optional UPDATE / DELETE

```
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -d banking -c \
"UPDATE public.logical_demo SET note='updated' WHERE id=1;"

sleep 2

kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c \
"SELECT * FROM public.logical_demo ORDER BY id;"
```

Step 9 — Cleanup

Drop the subscription first so the logical slot is removed on the publisher.

```
kubectl -n "$NS" exec "$SUB" -- psql -U postgres -d recoverydb -c "DROP SUBSCRIPTION IF EXISTS app_sub;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c "DROP PUBLICATION IF EXISTS app_pub;"
kubectl -n "$NS" exec "$PRIMARY" -- psql -U postgres -c \
"SELECT slot_name, slot_type, active FROM pg_replication_slots;"
```

If a logical slot remains, drop it on the primary. Leftover slots retain WAL and can fill the disk.

Troubleshooting

Symptom	Fix
wal_level is replica	Set logical in operator/Patroni and roll the cluster.
CREATE SUBSCRIPTION host not found	Use acid-postgres-ha.postgres-ha.svc.cluster.local or acid-postgres-ha.
replication / pg_hba rejected	Use repl_user with REPLICATION LOGIN and allow it on the Service.
could not create replication slot	Raise max_replication_slots / max_wal_senders; drop unused slots.
table does not exist on subscriber	Run the same CREATE TABLE on recoverydb first.
srsbstate stuck on i	Check apply worker pid, publisher logs, and network to the Service.
INSERT on primary never appears	Confirm publication lists logical_demo and subscription is enabled.

What this lab is not

This is not Patroni HA and not a failover drill. Physical streaming still protects acid-postgres-ha-1/2. Logical replication feeds recoverydb for selected tables only. DDL is not replicated. After a real failover, point the subscription at the Service and use a Patroni permanent logical slot so the bookmark survives promotion.